JAVA PROGRAMMING

Whatis Java?

- **Java is a programming language** and **platform**. Java is a high level, robust, object-oriented and secure programming language.
- Java was developed by Sun Microsystems (which is now the subsidiary of Oracle) in the year 1995. James Gosling is known as the father of Java.
- **Platform**: Any hardware or software environment in which a program runs, is known as a platform. Since Java has a runtime environment (JRE) and API, it is called a platform.

TIP

- Java is platform independent (Write Once, Run Anywhere).
- It follows the principle of Object-Oriented Programming.

Application

1. Desktop Applications such as acrobat reader, media player, antivirus, etc.
2. Web Applications such as irctc.co.in, javatpoint.com, etc.
3. Enterprise Applications such as banking applications.
4. Mobile
5. Embedded System
6. Smart Card
7. Robotics
8. Games, etc.

Features of Java

Simple

Java is very easy to learn, and its syntax is simple, clean and easy to understand. According to Sun Microsystems, Java language is a simple programming language because:

- Javasyntax is based on C++ (so easier for programmers to learn it after C++).
- Java has removed many complicated and rarely-used features, for example, explicit pointers, operator overloading, etc.
- There is no need to remove unreferenced objects because there is an Automatic Garbage Collection in Java.

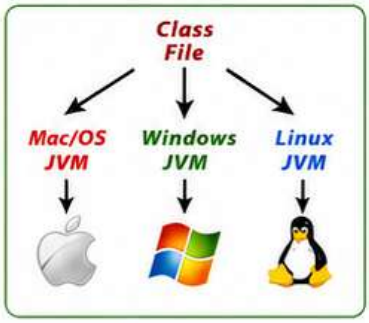


Object-oriented

Java is an object-oriented programming language. Everything in Java is an object. Object-oriented means we organize our software as a combination of different types of objects that incorporate both data and behaviour.

Platform Independent

Java is platform independent because it is different from other languages like C, C++, etc. which are compiled into platform specific machines while Java is a write once, run anywhere language. A platform is the hardware or software environment in which a program runs.



Secured

Java is best known for its security. With Java, we can develop virus-free systems. Java is secured because:

- **No explicit pointer**
- **Java Programs run inside a virtual machine sandbox**



Robust

The English meaning of Robust is strong. Java is robust because:

- It uses strong memory management.
- There is a lack of pointers.
- It performs garbage collection.

NOTE: Java is widely used in various applications and industries due to its robustness, security and platform independence.

NOTE: If you want any material follow by PYTHON_WORLD_IN Instagram page all study materials and exam pattern questions and answers AVAILABLE !!

JAVA PROGRAMMING

- Itusesstrongmemorymanagement.
- Thereisalackofpointersthatavoidsecurityproblems.
- Java provides automatic garbage collection which runs on the Java VirtualMachinetogetridofobjectswhicharenotbeingusedbyaJava application anymore.

TIP

These features make Java powerful, secure and widely used in different platforms.

Architecture-natural

Java is architecture natural because there are no implementation dependent features, for example, the size of primitive types is fixed.

Portable

Java is portable because it facilitates you to carry the Java bytecode to any platform. It doesn't require any implementation.

High-performance

Java is faster than other traditional interpreted programming languages because Java bytecode is "close" to native code.

Distributed

Java is distributed because it facilitates users to create distributed applications in Java.

Multi-threaded

A thread is like a separate program, executing concurrently. We can write Java programs that deal with many tasks at once by defining multiple threads.

Dynamic

Java is a dynamic language. It supports the dynamic loading of classes. It means classes are loaded on demand. It also supports functions from its native languages, i.e., C and C++.

NOTE: Java is widely used in various applications and industries due to its robustness, security and platform independence.

NOTE: If you want any material follow by PYTHON_WORLD_IN Instagram page all study materials and exam pattern questions and answers AVALIABLE !!

JAVA PROGRAMMING

FirstJavaProgram

- **class** keyword is used to declare a class in Java.
- **public** keyword is an access modifier that represents visibility. It means it is visible to all.
- **static** is a keyword. If we declare any method as static, it is known as the static method. The core advantage of the static method is that there is no need to create an object to invoke the static method. The main() method is executed by the JVM, so it doesn't require creating an object to invoke the main() method. So, it saves memory.
- **void** is the return type of the method. It means it doesn't return any value.
- **main** represents the starting point of the program.
- **String[] args or String args[]** is used for command line argument.
- **System.out.println()** is used to print statement. Here, System is a class, out is an object of the PrintStream class, println() is a method of the PrintStream class.

TIP

- main() is the entry point of every Java program.
- All Java programs are case-sensitive.
- Always end statements with semicolon (;).

What happens at runtime?

At runtime, the following steps are performed:



STEP EXPLANATION

1. The .java source file is compiled by the compiler to generate the .class file.
2. The classloader loads the class file into the memory.
3. The bytecode verifier verifies the bytecode for safety and correctness.
4. The interpreter interprets the bytecode line by line.
5. The runtime system provides the environment to execute the program.
6. Finally, the program runs on the hardware system.

NOTE: Java is widely used in various applications and industries due to its robustness, security and platform independence.

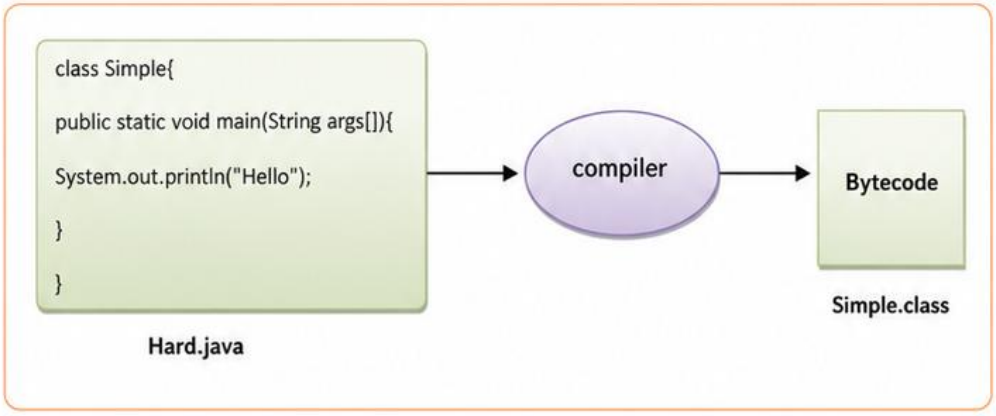
NOTE: If you want any material follow by PYTHON_WORLD_IN Instagram page all study materials and exam pattern questions and answers AVALIABLE !!

CREATED BY PYTHON_WORLD_IN

JAVA PROGRAMMING

Q) Can you save a Java source file by another name than the class name?

Yes, if the class is not public. It is explained in the figure given below:

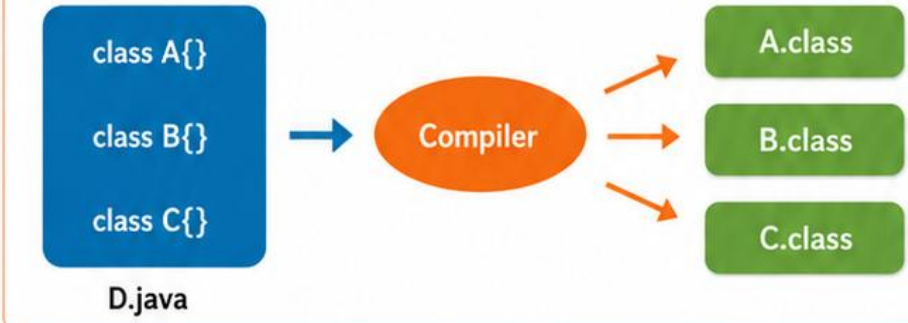


TIP

- If a class is not public, the file name and class name can be different.
- But if the class is public, the file name must be same as the class name.

Q) Can you have multiple classes in a java source file?

Yes, like the figure given below illustrates:



Difference between JDK,JRE, and JVM

JVM

JVM (Java Virtual Machine) is an abstract machine.
It is called a virtual machine because it doesn't physically exist.
It is a specification that provides a runtime environment.



NOTE: Java is widely used in various applications and industries due to its robustness, security and platform independence.

NOTE: If you want any material follow by PYTHON_WORLD_IN Instagram page all study materials and exam pattern questions and answers AVALIABLE !!

JAVA PROGRAMMING

JRE

JRE is an acronym for Java Runtime Environment. It is also written as Java RTE. The Java Runtime Environment is a set of software tools which are used for developing Java applications. It is used to provide the runtime environment.



JDK

JDK is an acronym for Java Development Kit. The Java Development Kit (JDK) is a software development environment which is used to develop Java applications and applets. It physically exists. It contains JRE + development tools.



TIP

- JRE is for running Java programs.
- JDK is for developing Java programs.
- JDK includes JRE + compiler + other tools.

Java Variables

A variable is a container which holds the value while the Java program is executed. A variable is assigned with a data type.

A variable is a name of memory location. There are three types of variables in java: local, instance and static.

1) Local Variable

A variable declared inside the body of the method is called local variable. You can use this variable only within that method and the other methods in the class aren't even aware that the variable exists.

A local variable cannot be defined with "static" keyword.

```
void display() {
    int x = 10;   // local variable
    System.out.println(x);
}
```

2) Instance Variable

A variable declared inside the class but outside the body of the method, is called an instance variable. It is not declared as static.

```
class Student {
    int id;          // instance variable
    String name;
    void setData() { ... }
}
```

3) Static Variable

A variable that is declared as static is called a static variable. It cannot be local. You can create a single copy of the static variable and share it among all the instances of the class.

```
class Counter {
    static int count = 0;  // static variable
    Counter() {
        count++;
    }
}
```

NOTE: Java is widely used in various applications and industries due to its robustness, security and platform independence.

NOTE: If you want any material follow by PYTHON_WORLD_IN Instagram page all study materials and exam pattern questions and answers AVALIABLE !!

CREATED BY PYTHON_WORLD_IN

JAVA PROGRAMMING

Data Types in Java

Data types specify the different sizes and values that can be stored in the variable. There are two types of data types in Java:

1. **Primitive data types:** The primitive data types include boolean, char, byte, short, int, long, float and double.
2. **Non-primitive data types:** The non-primitive data types include Classes, Interfaces, and Arrays.

TIP

- Primitive types store simple data values.
- Non-primitive types store complex data and objects.

Unicode System

Unicode is a universal international standard character encoding that is capable of representing most of the world's written languages.

Why Java uses Unicode System?

Before Unicode, there were many language standards:

- **ASCII** (American Standard Code for Information Interchange) for the United States.
- **ISO8859-1** for Western European Language.
- **KOI-8** for Russian.
- **GB18030** and **BIG-5** for Chinese, and soon.

Problem

This caused two problems:

1. A particular code value corresponds to different letters in the various language standards.
2. The encodings for languages with large character sets have variable length.
3. Some common characters are encoded as single bytes, others require two or more bytes.

NOTE: Java is widely used in various applications and industries due to its robustness, security and platform independence.

NOTE: If you want any material follow by **PYTHON_WORLD_IN** Instagram page all study materials and exam pattern questions and answers **AVALIABLE !!**

JAVA PROGRAMMING

Solution

To solve these problems, a new language standard was developed i.e. **Unicode System**.

In Unicode, a character holds 2 byte, so Java also uses 2 byte for characters.

- **lowest value :** \u0000
- **highest value :** \uFFFF

TIP

- Unicode supports more than 1 lakh characters.
- It is platform independent and supports every language.

Operators in Java

Operator in **Java** is a symbol that is used to perform operations.
For example: +, -, *, / etc.

There are many types of operators in Java which are given below:

- Unary Operator
- Arithmetic Operator
- Shift Operator
- Relational Operator
- Bitwise Operator
- Logical Operator
- Ternary Operator and
- Assignment Operator



Java Keywords

Java keywords are also known as reserved words. Keywords are particular words that act as a key to a code.
These are predefined words by Java so they cannot be used as a variable or object name or class name.



NOTE: Java is widely used in various applications and industries due to its robustness, security and platform independence.

NOTE: If you want any material follow by **PYTHON_WORLD_IN** Instagram page all study materials and exam pattern questions and answers **AVALIABLE !!**

JAVA PROGRAMMING

Java Control Statements | Control Flow in Java

Java provides three types of control flow statements.

1 Decision Making Statements
- if statements
- switch statement

2 Loop Statements
- do while loop
- while loop
- for loop
- for-each loop

3 Jump Statements
- break statements
- continue statement

TIP

- Control statements control the flow of execution in a program.
- They help to make decisions, repeat tasks and transfer control.

Decision-Making Statements:

As the name suggests, decision-making statements decide which statement to execute and when.

1 if Statement:

In Java, the "if" statement is used to evaluate a condition. The control of the program is diverted depending upon the specific condition.
The condition of the if statement gives a Boolean value, either true or false.

```
if (condition) {
    // block of code
}
```

2 if-else Statement:

The if-else statement is an extension to the if statement, which uses another block of code, i.e., else block.

```
if (condition) {
    // block of code
} else {
    // else block of code
}
```

3 if-else-if Ladder:

The if-else-if statement contains the if statement followed by multiple else-if statements. In other words, we can say that it is the chain of if-else.

```
if (condition1) {
    // block of code
} else if (condition2) {
    // block of code
} else if (condition3) {
    // block of code
} else {
    // else block of code
}
```

NOTE: Java is widely used in various applications and industries due to its robustness, security and platform independence.

NOTE: If you want any material follow by **PYTHON_WORLD_IN** Instagram page all study materials and exam pattern questions and answers **AVALIABLE !!**

CREATED BY PYTHON_WORLD_IN

JAVA PROGRAMMING

Statements that create a decision tree where the program may enter in the block of code where the condition is true.

4) Nested if-statement

In nested if-statements, the if statement can contain a if or if-else statement inside another if or else-if statement.

```
if (condition1) {
    if (condition2) {
        // block of code
    } else {
        // else block of code
    }
}
```

Switch Statement

In Java, **Switch** statements are similar to **if-else-if** statements. The **switch** statement contains multiple blocks of code called cases and a single case is executed based on the variable which is being switched.

```
switch (variable) {
    case value1:
        // block of code
        break;
    case value2:
        // block of code
        break;
    default:
        // default block
}
```

TIP

- Decision-making statements are used to make decisions.
- Loop statements help in executing repetitive tasks efficiently.

Loop Statements

In programming, sometimes we need to execute the block of code repeatedly while some condition evaluates to true. However, loop statements are used to execute the set of instructions in a repeated order.

1. for loop
2. while loop
3. do-while loop

for Loop in Java

In Java, for loop is similar to C and C++. It enables us to initialize the loop variable, check the condition, and increment/decrement in a single line of code. We use the for loop only when we exactly know the number of times, we want to execute the block of code.

```
for (int i = 1; i <= n; i++) {
    // block of code
}
```

for-each Loop in Java

Java provides an enhanced for loop to traverse the data structures like array or collection. In the for-each loop, we don't need to update the loop variable. The syntax to use the for-each loop in Java is given below.

```
for (data_type var : array_name/collection_name) {
    // statements
}
```

NOTE: Java is widely used in various applications and industries due to its robustness, security and platform independence.

NOTE: If you want any material follow by PYTHON_WORLD_IN Instagram page all study materials and exam pattern questions and answers AVALIABLE !!

CREATED BY PYTHON_WORLD_IN

JAVA PROGRAMMING

Java while Loop

The **while loop** is also used to iterate over the number of statements multiple times. However, if we don't know the number of iterations in advance, it is recommended to use a while loop.

```
int i = 1;
while(i <= 10) {
    System.out.println(i);
    i++;
}
```

Java do-while Loop

The **do-while loop** checks the condition at the end of the loop after executing the loop statements. When the number of iterations is not known and we have to execute the loop at least once, we can use do-while loop.

It is also known as the exit-controlled **loop** since the condition is not checked in advance. The syntax of the do-**while loop** is given below.

```
int i = 1;
do {
    System.out.println(i);
    i++;
} while(i <= 10);
```

TIP

- Loops help repeat tasks efficiently.
- Jump statements help control the flow of execution in a program.

Jump Statements

Jump statements are used to transfer the control of the program to the specific statements. In other words, jump statements transfer the execution control to the other part of the program.

Java break Statement

As the name suggests, the **break statement** is used to break the current flow of the program and transfer the control to the next statement outside a loop or switch statement.

```
for(int i = 1; i <= 10; i++) {
    if(i == 5) {
        break;    // exit the loop
    }
    System.out.println(i);
}
```

Java continue Statement

Unlike break statement, the **continue statement** doesn't break the loop, whereas, it skips the specific part of the loop and jumps to the next iteration of the loop immediately.

```
for(int i = 1; i <= 10; i++) {
    if(i == 5) {
        continue;   // skip this iteration
    }
    System.out.println(i);
}
```

NOTE: Java is widely used in various applications and industries due to its robustness, security and platform independence.

NOTE: If you want any material follow by **PYTHON_WORLD_IN** Instagram page all study materials and exam pattern questions and answers **AVALIABLE !!**

JAVA PROGRAMMING

Java Object Class

Java OOPs Concepts

Object-Oriented Programming is a paradigm that provides many concepts, such as **inheritance, data binding, polymorphism**, etc.

TIP

- OOPs makes code modular, reusable and easy to maintain.
- It is used in almost every Java application.

1. Object

Any entity that has state and behaviour is known as an object.
For example, a chair, pen, table, keyboard, bike, etc.
It can be physical or logical.



Examples: Chair, Pen, Table, Bike

2. Class

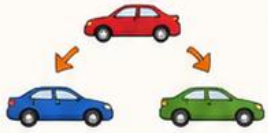
Collection of objects is called class.
It is a logical entity or a blueprint.



Example:
Class Student

3. Inheritance

When one object acquires all the properties and behaviours of a parent object, it is known as inheritance.
It provides code reusability.



Example:
Car is inherited by Audi, BMW.

4. Polymorphism

If one task is performed in different ways,
it is known as polymorphism.
For example: to convince the customer differently,
to draw something, e.g., shape, triangle, rectangle, etc.



Example:
Shape can be drawn in many forms.

5. Abstraction

Hiding internal details and showing functionality
is known as abstraction.
For example, in phone call, we don't know
the internal processing.



Example:
Using a phone call without knowing how it works inside.

6. Encapsulation

Binding (or wrapping) code and data together into
a single unit is known as encapsulation.
For example, a capsule, it is wrapped with
different medicines.



Example:
Capsule is wrapped with medicines.

NOTE: Java is widely used in various applications and industries due to its robustness, security and platform independence.

NOTE: If you want any material follow by **PYTHON_WORLD_IN** Instagram page all study materials and exam pattern questions and answers **AVALIABLE !!**

CREATED BY PYTHON_WORLD_IN

JAVA PROGRAMMING

Coupling

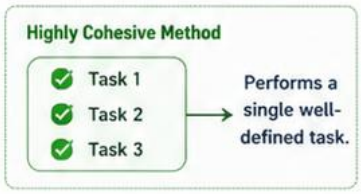
Coupling refers to the knowledge or information or dependency of another class. It arises when classes are aware of each other.



Class A is aware of Class B and vice versa.

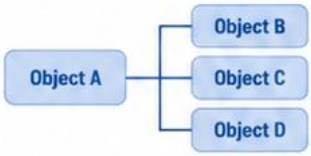
Cohesion

Cohesion refers to the level of a component which performs a single well-defined task. A single well-defined task is done by a highly cohesive method.



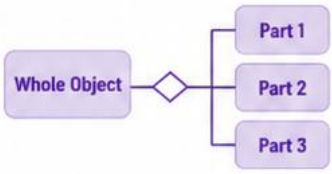
Association

Association represents the relationship between the objects. Here, one object can be associated with one object or many objects.



Aggregation

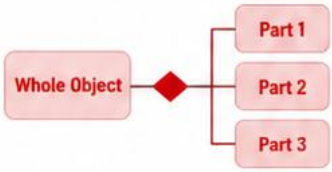
Aggregation is a way to achieve Association. Aggregation represents the relationship where one object contains other objects as a part of its state.





Composition

The composition is also a way to achieve Association. The composition represents the relationship where one object contains other objects as a part of its state.



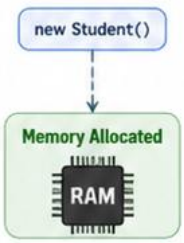


Constructors in Java

In Java, a constructor is a block of code similar to the method. It is called when an instance of the class is created. At the time of calling constructor, memory for the object is allocated in the memory.

```
class Student {
   int id;
   String name;

   // Constructor
   Student() {
      id = 0;
      name = "Unknown";
   }
}
```



Constructor Overloading in Java

Constructor overloading in Java is a technique of having more than one constructor with different parameter lists.

```
class Student {
    Student() { }
    Student(int id) { }
    Student(int id, String name) { }
}
```

Multiple Constructors

- No parameter
- One parameter
- Multiple parameters

NOTE: Java is widely used in various applications and industries due to its robustness, security and platform independence.

NOTE: If you want any material follow by **PYTHON_WORLD_IN** Instagram page all study materials and exam pattern questions and answers **AVALIABLE !!**

TIP

- OOP makes programs modular, reusable and easy to maintain.
- It improves code reusability and reduces complexity.

CREATED BY PYTHON_WORLD_IN

JAVA PROGRAMMING

Singleton Design Pattern in Java

Singleton Pattern says that just **"define a class that has only one instance and provides a global point of access to it"**.

In other words, a class must ensure that only single instance should be created and single object can be used by all other classes.

TIP

- Ensures only one instance of a class exists.
- Provides a global access point to that instance.
- Useful for logging, caching, configuration, database connections, etc.

There are two forms of singleton design pattern

1 Early Instantiation

Creation of instance at load time.

Class Loaded → Instance Created

2 Lazy Instantiation

Creation of instance when required.

Request for Instance → Instance Created

Advantages of Singleton Design Pattern

Saves memory because object is created at each request.

Only single instance is reused again and again.

Private Constructor in Java

Java allows us to declare a constructor as private. We can declare a constructor private by using the **private** access specifier. Note that if a constructor is declared private, we are not able to create an object of the class.
Instead, we can use this private constructor in **Singleton Design Pattern.**

```
class Example {
    private Example() {
        // private constructor
    }
}
```

Rules for Private Constructor

1	It does not allow a class to be sub-classed.
2	It does not allow to create an object outside the class.
3	If a class has a private constructor and when we try to extend the class, a compile-time error occurs.
4	We cannot access a private constructor from any other class.
5	If all the constant methods are there in our class, we can use a private constructor.

```
class A {
    private A() { }
}
class B extends A { }
//  Compile-time Error
```

```
class Utils {
    private Utils() { }

    public static int add(int a, int b) {
        return a + b;   // constant method
    }
}
// Used without creating object
int sum = Utils.add(10, 20);
```

NOTE: Java is widely used in various applications and industries due to its robustness, security and platform independence.

NOTE: If you want any material follow by **PYTHON_WORLD_IN** Instagram page all study materials and exam pattern questions and answers **AVALIABLE !!**

CREATED BY PYTHON_WORLD_IN

JAVA PROGRAMMING

Use Cases of Private Constructor

The main purpose of using a private constructor is **to restrict object creation**. We use private constructors to implement the singleton design pattern. The use-cases of the private constructor are as follows:

- It can be used with static members-only classes.
- It can be used with **static utility** or constant classes.
- It can also be used to create **singleton** classes.
- It can be used to assign a name, for instance creation by utilizing **factory methods**.
- It is also used to avoid sub-classing.

TIP

- Private constructor prevents the object from being created outside the class.
- It provides more control over object creation.
- It is a key concept in design patterns like Singleton.

Difference b/w Constructor and Method

Constructor		Method
A constructor is used to initialize the state of an object.	1	A method is used to expose the behavior of an object.
A constructor must not have a return type.	2	A method must have a return type.
The constructor is invoked implicitly.	3	The method is invoked explicitly.
The Java compiler provides a default constructor if you don't have any constructor in a class.	4	The method is not provided by the compiler in any case.
The constructor name must be same as the class name.	5	The method name may or may not be same as class name.

Java static keyword

The static keyword in Java is used for memory management mainly. We can apply static keyword with variables, methods, blocks and nested classes.

Variables | Methods | Blocks | Nested Classes

NOTE: Java is widely used in various applications and industries due to its robustness, security and platform independence.

NOTE: If you want any material follow by PYTHON_WORLD_IN Instagram page all study materials and exam pattern questions and answers AVALIABLE !!

CREATED BY PYTHON_WORLD_IN

JAVA PROGRAMMING

The static keyword belongs to the class than an instance of the class.

TIP

- static means "shared among all objects".
- It helps to save memory and improve performance.
- It is used for memory management mainly.

1 Java static variable

If you declare any variable as static, it is known as static variable.

- The static variable can be used to refer to the common property of all objects (which is not unique for each object), for example, the company name of employees, college name of students, etc.



- The static variable gets memory only once in the class area at the time of class loading.



2 Java static method

If you apply static keyword with any method, it is known as static method.

- A static method belongs to the class rather than the object of a class.
- A static method can be invoked without the need for creating an instance of a class.
- A static method can access static data member and can change the value of it.

Example

```
class MathUtil {
   static int add(int a, int b) {
      return a + b;
   }
}

// Calling without creating object
int result = MathUtil.add(10, 20);
```

Q Why is the Java main method static?

Ans: It is because the object is not required to call a static method. If it were a non-static method, JVM creates an object first then call main() method that will lead the problem of extra memory allocation.

Flow of Execution

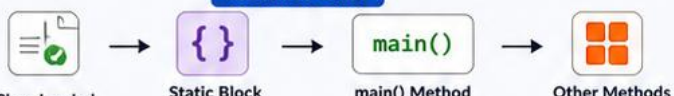


3 Java static block

A static block is used to initialize the static data member.

- It is used to initialize the static data member.
- It is executed before the main method at the time of class loading.

Execution Order



Syntax

```
class Example {
   static int data;

   static {
      data = 100;   // static block
   }

   public static void main(String[] args) {
      System.out.println(data);
   }
}
```

NOTE: Java is widely used in various applications and industries due to its robustness, security and platform independence.

NOTE: If you want any material follow by PYTHON_WORLD_IN Instagram page all study materials and exam pattern questions and answers AVALIABLE !!

CREATED BY PYTHON_WORLD_IN

JAVA PROGRAMMING

Can we execute a program without main() method?

Ans: No, one of the ways was the static block, but it was possible till JDK 1.6. Since JDK 1.7, it is not possible to execute a Java class without the main method.

TIP

- main() method is the entry point of every Java application.
- From JDK 1.7 onwards, main() is mandatory to run a program.

this keyword in Java

There can be a lot of usage of Java this keyword. In Java, this is a reference variable that refers to the current object.

Usage of Java this Keyword

There can be a lot of usage of Java this keyword. In Java, this is a reference variable that refers to the current object.

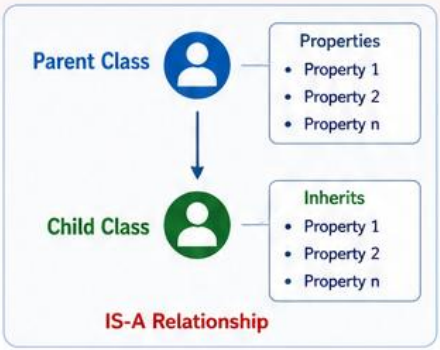
01. this can be used to refer current class instance variable.
02. this can be used to invoke current class method (implicitly).
03. this() can be used to invoke current class constructor.
04. this can be passed as an argument in the method call.
05. this can be passed as argument in the constructor call.
06. this can be used to return the current class instance from the method.

Java Inheritance

Inheritance in Java

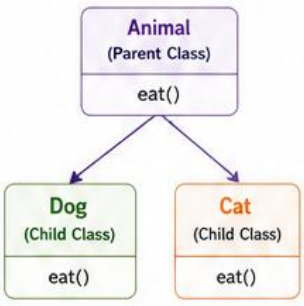
Inheritance in Java is a **mechanism** in which one object acquires all the **properties and behaviours** of a parent object.

Inheritance represents the **IS-A relationship** which is also known as a parent-child relationship.



Why use inheritance in Java?

- For Method Overriding (so runtime polymorphism can be achieved).




```java
class Animal {
   void eat() {
      System.out.println("Eating...");
   }
}

class Dog extends Animal {
   @Override
   void eat() {
      System.out.println("Dog is eating");
   }
}
```

NOTE: Java is widely used in various applications and industries due to its robustness, security and platform independence.

NOTE: If you want any material follow by PYTHON_WORLD_IN Instagram page all study materials and exam pattern questions and answers AVALIABLE !!

CREATED BY PYTHON_WORLD_IN

JAVA PROGRAMMING

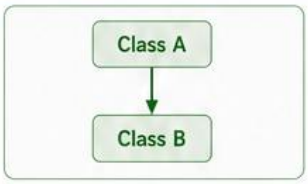
- For Code Reusability.

Types of Inheritance in Java

On the basis of class, there can be three types of inheritance in Java: **single, multilevel and hierarchical.**

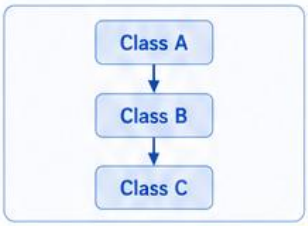
1 Single Inheritance

When a class inherits another class, it is known as single inheritance.



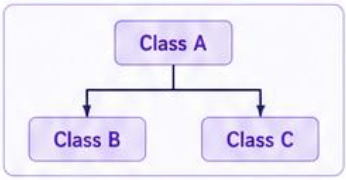
2 Multilevel Inheritance Example

When there is a chain of inheritance, it is known as multilevel inheritance.



3 Hierarchical Inheritance Example

When two or more classes inherit a single class, it is known as hierarchical inheritance.

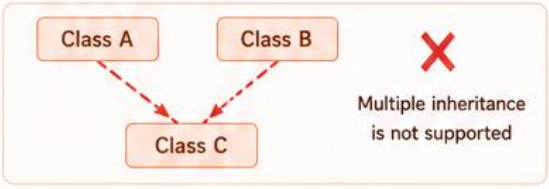


TIP

- Inheritance promotes code reusability.
- It improves the extensibility and maintainability of code.
- Aggregation shows a part-whole (HAS-A) relationship.

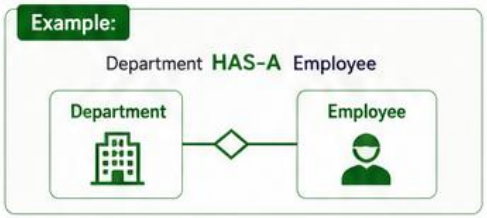
Why multiple inheritance is not supported in Java?

To reduce the complexity and simplify the language, multiple inheritance is not supported in Java.



Aggregation in Java

If a class has an entity reference, it is known as Aggregation. Aggregation represents **HAS-A** relationship.



Method Overloading in Java

If a class has multiple methods having same name but different in parameters, it is known as **Method Overloading.**

Example:

```
class Calculator {
   int add(int a, int b) { return a + b; }
   int add(int a, int b, int c) { return a + b + c; }
   double add(double a, double b) { return a + b; }
}
```

Calling Methods

```
add(10, 20);
add(10, 20, 30);
add(10.5, 20.5);

// All valid calls with different parameters
```

If we have to perform only one operation, having same name of the methods increases the **readability** of the program.

NOTE: Java is widely used in various applications and industries due to its robustness, security and platform independence.

NOTE: If you want any material follow by **PYTHON_WORLD_IN** Instagram page all study materials and exam pattern questions and answers **AVALIABLE !!**

CREATED BY PYTHON_WORLD_IN

JAVA PROGRAMMING

Q Why Method Overloading is not possible by changing the return type of method only?

Ans: In Java, method overloading is not possible by changing the return type of the method only because of ambiguity.

TIP

- Method overloading depends on method name and parameters.
- Return type is not considered.

Q Can we overload Java main() method?

Ans: Yes, by method overloading.

You can have any number of main methods in a class by method overloading.

But **JVM** calls main() method which receives **String array as arguments only.**

Example:

```
class Demo {
   public static void main(String[] args) {
      System.out.println("Main with String array");
   }
   public static void main(int a) {
      System.out.println("Main with int");
   }
   public static void main() {
      System.out.println("Main with no args");
   }
}
```

Java Polymorphism

Method Overriding in Java

If subclass (child class) has the same method as declared in the parent class, it is known as **method overriding in Java.**

In other words, if a subclass provides the specific implementation of the method that has been declared by one of its parent classes, it is known as method overriding.

Example:

```
class Animal {
   void sound() {
      System.out.println("Animal makes sound");
   }
}

class Dog extends Animal {
   @Override
   void sound() {
      System.out.println("Dog barks");
   }
}

// Calling overridden method
Animal a = new Dog();
a.sound();    // Output: Dog barks
```

Rules for Java Method Overriding



- STEP 01 — Method must have same name as in the parent class.
- STEP 02 — Method must have same parameters as in the parent class.
- STEP 03 — There must be IS-A relationship (inheritance).

NOTE: Java is widely used in various applications and industries due to its robustness, security and platform independence.

NOTE: If you want any material follow by **PYTHON_WORLD_IN** Instagram page all study materials and exam pattern questions and answers **AVALIABLE !!**

CREATED BY PYTHON_WORLD_IN

JAVA PROGRAMMING

Covariant Return Type

The covariant return type specifies that the **return type may vary** in the **same direction** as the subclass.

TIP

- Covariant return type is allowed in method overriding.
- The return type of the overriding method must be the same as or a subtype (child) of the return type of the overridden method.

Super Keyword in Java

The **super** keyword in Java is a reference variable which is used to refer immediate parent class object.

Usage of Super Keyword

1. super can be used to refer immediate parent class instance variable.
2. super can be used to invoke immediate parent class method.
3. super() can be used to invoke immediate parent class constructor.

Instance Initializer Block

Instance Initializer block is used to initialize the **instance data member**. It runs each time when object of the class is created.

Example

```
class Test {
   int x;

   // instance initializer block
   {
     x = 100;
     System.out.println("Instance block
      is executed");
   }

   Test() {
     System.out.println("Constructor");
   }
}
```

Final Keyword in Java

The **final** keyword in Java is used to **restrict** the user. The java final keyword can be used in many contexts. Final can be:

1. variable
2. method

Example

```
class Demo {
   final int MAX = 100;    // final variable

   final void show() {     // final method
      System.out.println("This is a final method");
   }
}
```

NOTE: Java is widely used in various applications and industries due to its robustness, security and platform independence.

NOTE: If you want any material follow by **PYTHON_WORLD_IN** Instagram page all study materials and exam pattern questions and answers **AVALIABLE** !!

JAVA PROGRAMMING

3 class

1 Java final variable

If you make any variable as final, you cannot change the value of final variable (It will be constant).

Java Final Keyword

- Stop Value Change
- Stop Method Overridding
- Stop Inheritance

TIP

final keyword is used to restrict modification and improve security.

2 Java final method

If you make any method as final, you cannot override it.

Cannot Override

A final method cannot be overridden by subclass.

3 Java final class

If you make any class as final, you cannot extend it.

Cannot Extend

A final class cannot be extended by any other class.

Q Is final method inherited? → **Ans:** Yes, final method is inherited but you **cannot** override it.

Q What is blank or uninitialized final variable? → **Ans:** A final variable that is not initialized at the time of declaration is known as blank final variable.

Q Can we initialize blank final variable? → **Ans:** Yes, but only in constructor.

Q What is final parameter? → **Ans:** If you declare any parameter as final, you cannot change the value of it.

NOTE: Java is widely used in various applications and industries due to its robustness, security and platform independence.

NOTE: If you want any material follow by PYTHON_WORLD_IN Instagram page all study materials and exam pattern questions and answers **AVAILABLE** !!

CREATED BY PYTHON_WORLD_IN

JAVA PROGRAMMING

Can we declare a constructor final?

Ans: No, because constructor is never inherited.

TIP

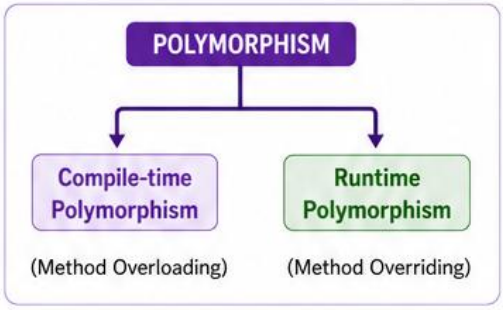
- Constructor cannot be declared final.
- Because constructors are not inherited by subclasses.

Polymorphism in Java

Polymorphism in Java is a concept by which we can perform a **single action in different ways.**

There are two types of polymorphism in Java: **compile-time polymorphism** and **runtime polymorphism.**

We can perform polymorphism in Java by method overloading and method overriding.



Runtime Polymorphism in Java

Runtime polymorphism or Dynamic Method Dispatch is a process in which a call to an overridden method is resolved at runtime rather than compile-time.

Upcasting

If the reference variable of Parent class refers to the object of Child class, it is known as upcasting.

Parent class reference → refers to → Child class object

(Upcasting)

Example: Runtime Polymorphism

```
class Animal {
  void sound() { System.out.println("Animal makes sound"); }
}
class Dog extends Animal {
  @Override
  void sound() { System.out.println("Dog barks"); }
}
class Test {
  public static void main(String[] args) {
    Animal a = new Dog();  // upcasting
    a.sound();             // Dog class method is called at runtime
  }
}
```

Animal a = new Dog(); (Upcasting) → a.sound() → Dog class sound() called at runtime

Static Binding and Dynamic Binding

Connecting a method call to the method body is known as binding. There are two types of binding:

1	Static Binding (Early Binding)	The method call is resolved at compile-time.
2	Dynamic Binding (Late Binding)	The method call is resolved at runtime.

Feature	Static Binding (Early Binding)	Dynamic Binding (Late Binding)
When resolved?	Compile-time	Runtime
Also known as	Early Binding	Late Binding
Achieved by	Method Overloading	Method Overriding
Perfermance	Faster	Slower
Example	Overloaded methods	Overridden methods

NOTE: Java is widely used in various applications and industries due to its robustness, security and platform independence.

NOTE: If you want any material follow by **PYTHON_WORLD_IN** Instagram page all study materials and exam pattern questions and answers **AVALIABLE !!**

CREATED BY PYTHON_WORLD_IN

JAVA PROGRAMMING

Static vs Dynamic Binding

Static Binding — When type of the object is determined at compile time, it is known as static binding.

Dynamic Binding — When type of the object is determined at run-time, it is known as dynamic binding.

TIP

- Static binding happens at compile time.
- Dynamic binding happens at runtime.
- Dynamic binding is also known as late binding.

Static Binding

When type of the object is determined at compiled time (by the compiler), it is known as static binding.

Dynamic Binding

When type of the object is determined at run-time, it is known as dynamic binding.

Java instanceof Operator

The **instanceof** operator is used to test whether the object is an instance of the specified type (class or subclass or interface).

The **instanceof** in Java is also known as type comparison operator because it compares the instance with type. It returns either true or false.

Example

```
class A {}
class B extends A {}

A obj = new B();

System.out.println(obj instanceof A);   // true
System.out.println(obj instanceof B);   // true
System.out.println(obj instanceof Object);  // true
System.out.println(obj instanceof String);  // false
```

Java Abstraction

Abstract Class in Java

A class which is declared with the **abstract** keyword is known as an **abstract** class in Java.

It can have **abstract** and non-abstract methods (methods with body).

Example: Abstract Class

```
abstract class Shape {
   abstract void draw();     // abstract method (no body)

   void color() {            // non-abstract method (with body)
       System.out.println("Color of shape");
   }
}
```

NOTE: Java is widely used in various applications and industries due to its robustness, security and platform independence.

NOTE: If you want any material follow by PYTHON_WORLD_IN Instagram page all study materials and exam pattern questions and answers AVALIABLE !!

CREATED BY PYTHON_WORLD_IN



Website: https://tinyurl.com/Pythonworldin

WhatsApp: 9771025313